Synopsis

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Unit of MAHE, Manipal

WardWatch

A Responsive Civic-Tech Framework for Community Infrastructure Management

By Asmita Guha, Reg No.: 230911112, IT-D, Roll No.: 19

1. Introduction

WardWatch is a cross-platform application that helps local residents communicate with their municipal representatives and is driven by data. Residents are able to report problems related to their infrastructure like road issues, cleanliness, and electricity problems. Residents are also able to see what is being done to resolve the issues with the help of real-time tracking. WardWatch enables the local authorities to have a data-driven approach to solve community issues that are regularly reported. This is done with the power of modern web tools and persistent cloud storage.

2. Problem Statement

Traditional methods of civic reporting and complaints about infrastructure issues posted on social media are not tangible, unified, or systematic. This makes the problems fall into a feedback black hole. This goes on for a long time, and the local authorities don't have an organized system to see what issues to solve. This is a perfect example of a system that lacks persistence and verifiable systems to track the life cycle of problems that were reported until the problems are resolved.

3. Objectives

- **Platform Versatility:** To develop a responsive web-first application using **Flutter** that ensures accessibility across desktops, tablets, and smartphones.
- **Data Integrity & Persistence:** To use **Supabase (PostgreSQL)** or **Firebase (Firestore)** to store data at the backend for secure, long-term data storage.
- **Operational Transparency:** To establish a transparent role-based workflow (Resident vs. Admin) that tracks the status of reports through various stages: *Submitted, Acknowledged, In-Progress,* and *Resolved*.
- **Evidence-Based Governance:** To provide a multimedia-supported reporting tool that allows users to upload photographic evidence, ensuring higher quality and accuracy in reported data.

4. Technical Specifications

The tech stack is selected for its scalability, industry-standard security, and high performance-to-cost ratio:

Layer	Primary Technology	Description
Frontend UI/UX	**Flutter (Dart)**	Leverages a single codebase for a responsive, high-performance web experience.
Backend-as-a-Service	**Supabase / Firebase**	**Supabase (Postgres):** High-integrity relational mapping for complex queries. **Firebase (NoSQL):** Low-latency document synchronization.
Persistent Storage	**S3-Compatible Storage**	Cloud-based hosting for multimedia assets (Supabase Buckets or Firebase Storage).
Authentication	**OAuth 2.0 / JWT**	Secure, industry-standard user identity management and role-based access control.
CI/CD & Hosting	**Vercel / Firebase Hosting**	Automated deployment pipelines for high-availability web hosting.

5. Future Scope & Research Potential

The objective of WardWatch's architectural design is to provide a basis for the consideration of scalable design principles for urban computing and civic-tech research perspectives, the researching of which could develop into wider programs of activity described as 'urban computing'. The possibilities for this direction may be as follows:

- **Empirical Case Studies:** A deployment may be made of the platform for localised pilot case studies in municipalities within specific wards where longitudinal data can

be gathered about resident engagement and the efficacy of digital feedback devices to assist with local governance.

- **Performance Analytics:** The aggregate data collected through the system allows for comparative analyses of municipal response times; such research would be able to support the discussion around and support the impact of crowdsourced data on the efficiency of municipalities, and would encourage efforts to present this research at international forums such as IEEE Smart Cities.
- **Computer Vision Integration:** To enhance the validating process, future iterations of WardWatch may explore the possible benefits of integrating deep learning (CNN) models into the validation process where the system would (1) provide categorisation of issues based on user-submitted images and (2) allow for a preliminary assessment of issues with the level of severity ascribed to them; this work will reduce the number of manual administrative triage activities required by providing an easier way to assess issues based on image submissions.
- **Data-Driven Governance Models:** The structured datasets produced as a result of using the platform may, with time, become an initial working foundation for the prediction of maintenance of critical infrastructure by assisting municipalities in identifying infrastructure at risk of failure before critical failures occur.

6. Future Expansion

- **GIS Integration:** Google Maps APIs embedded into the application providing a “heatmap,” allowing authorities to see clusters of high-needs areas.
- **Multilingual Support:** Expanding UI to include local languages to accommodate all residents in the ward, creating an inclusive environment.
- **Gamification Engine:** Implementing a Community Contributor Leaderboard so that residents are encouraged to report issues and to verify others’ work, resulting in a sense of community pride.
- **Automated Notifications:** Integrated with Twilio or SendGrid to send automated SMS/Email notifications to residents as their reported problems progress through the resolution pipeline.

Roadmap

Roadmap of Project

🏗️ 1. Project Directory Structure

In Flutter, organizing your files by **Feature** (rather than just putting all screens in one folder) is the industry standard. This makes your project scalable.

```
Plaintext

wardwatch/
├── lib/
│   ├── main.dart              # App entry point & Supabase initialization
│   ├── core/                  # Constants, themes, and global utilities
│   │   ├── constants.dart     # API keys, colors, and strings
│   │   └── theme.dart         # Material 3 design system config
│   ├── services/              # Backend logic
│   │   ├── auth_service.dart  # Login, Signup, Logout
│   │   └── database_service.dart # CRUD operations (fetch/post reports)
│   ├── models/                # Data classes (JSON to Dart objects)
│   │   ├── report_model.dart
│   │   └── user_model.dart
│   └── views/                 # UI Screens
│       ├── common/            # Landing, Auth (Login/Signup/Forgot Pass)
│       ├── resident/          # Home, Reporting Form, Detail View
│       └── admin/             # Dashboard, Master Queue, Analytics
├── assets/                    # Images and Icons
└── pubspec.yaml               # Project dependencies
```

🏛️ 2. Finalized Database Structure (Supabase)

This structure is normalized to **3rd Normal Form (3NF)** and includes the security layer for "passwords and stuff."

Table Name	Role	Essential Columns	Logic / Relationship
`auth.users`	**Auth System**	`id` (UUID), `email`, `encrypted_password`	**System Managed:** Securely stores credentials.

profiles	**Identity**	id (UUID), full_name, email, role, phone_number	**Primary Profile:** Linked to auth.users.id.
categories	**Classification**	id, category_name, default_priority, sla_days	**Master Data:** Sets baseline urgency/SLA.
reports	**Core Ticket**	id, user_id (FK), cat_id (FK), title, desc, priority_level, status	**Main Ledger:** Tracks specific community issues.
locations	**Spatial Data**	report_id (FK), latitude, longitude, landmark_details	**Geo-Mapping:** 1:1 link with reports.
media	**Persistence**	id, report_id (FK), url, media_type	**Asset Links:** Points to Supabase Storage files.
status_logs	**Research Log**	id, report_id (FK), status_from, status_to, changed_at, remarks	**Audit Trail:** Captures history for research.

📱 3. Application Views (Total: 8 Views)

To provide a complete experience, we need these specific screens:

1. **Landing View:** General pitch with "Report Now" and "Admin Login" entry points.
2. **Auth Views (Login / Sign Up / Forgot Password):** Handling session entry and password recovery via Supabase Auth.
3. **Resident Dashboard:** A feed of the user's active reports and a button to launch the wizard.
4. **Reporting Wizard:** A multi-step form (Category -> Details/Location -> Photo Upload).

5. **Report Detail (Timeline):** A view showing the issue progress using a vertical timeline from `status_logs`.
6. **Admin Dashboard:** High-level analytics (Pie charts of categories, Bar charts of fix times).
7. **Admin Master Queue:** A filterable data table for the Councillor to manage all ward issues.
8. **Admin Action Screen:** Where the Admin reviews evidence and updates the status/priority.

🔄 4. Integration & Data Flow

How Supabase works with Flutter:

1. **Auth:** Flutter calls `supabase.auth.signIn()`. If successful, a JWT (Token) is stored locally.
2. **Trigger:** When a new user signs up, a **SQL Trigger** in the backend automatically populates the `profiles` table with their name and email.
3. **Storage:** Photos are sent to a **Supabase Bucket**. The app receives a URL, which is then saved in the `media` table.
4. **Real-time:** We will use **StreamBuilder** in Flutter so that when the Admin updates a status, the Resident's screen updates instantly without a refresh.

🛠️ 5. Phase-by-Phase Roadmap

Phase 1: The Foundation (Week 1)

- Initialize Flutter Web project.
- Setup Supabase project & run the SQL schema.
- Create the `AuthService` class to handle Login/Signup.

Phase 2: The Resident Flow (Week 2-3)

- Build the **Reporting Wizard**.
- Integrate `image_picker` for photos and `geolocator` for coordinates.
- Implement the "My Reports" feed using a `FutureBuilder`.

Phase 3: The Admin Flow (Week 4)

- Build the **Master Queue** with filtering logic.
- Implement the status update function that writes to `status_logs`.
- Build the simple Analytics dashboard using `fl_chart`.

Phase 4: Vast Database Seeding (Week 5)

- I will provide you with a **Python Script** or a **Mass SQL Insert Script**.

- This will generate **50+ realistic entries** with varied Manipal coordinates, categories, and timestamps (some resolved, some pending) to make your research graphs look impressive.

Phase 5: Final Polish & Deployment (Final Week)

- Add Material 3 "Loading Shimmer" effects.
- Deploy to **Vercel** or **Firebase Hosting**.
- Prepare the final PDF documentation using the formal synopsis we created.

Database Structure

The system utilizes a Normalized Relational Schema (3NF) to ensure data integrity and provide a longitudinal dataset for research.

Table Name	Role	Essential Columns	Logic / Relationships
auth.users	**Auth System**	id (UUID), email, encrypted_password	**Hidden System Table:** Managed by Supabase to securely store credentials and handle password hashing.
profiles	**Identity**	id (UUID), full_name, email, role, phone_number	**Primary Profile:** Linked to auth.users.id. Acts as the central identity record for residents and admins.
categories	**Classification**	id, category_name, default_priority, sla_days	**Master Data:** Defines issue types. default_priority is used to auto-populate report urgency upon creation.
reports	**Core Ticket**	id, user_id (FK), category_id (FK), title, description, priority_level, current_status	**Main Ledger:** Tracks specific issues. priority_level starts as a system default but is editable by admins.
locations	**Spatial Data**	report_id (FK), latitude, longitude, landmark_details	**Geo-Mapping:** 1:1 relationship with reports. Ensures every issue has a precise physical coordinate.

media	**Persistence**	id, report_id (FK), url, media_type (Image/Video)	**Asset Links:** Stores URLs for files in Cloud Storage. Allows for multiple pieces of evidence per report.
status_logs	**Research Log**	id, report_id (FK), status_from, status_to, changed_at, remarks	**Audit Trail:** Captures every status transition for time-to-resolution analysis in the final research paper.